

```

def get_neighbors(state):
    """Finds all valid adjacent states by moving the blank space
    (0)."""
    neighbors = []
    blank_idx = state.index(0)
    row, col = divmod(blank_idx, 3)

    moves = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1,
    'Right')]

    for dr, dc, move_name in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank_idx = new_row * 3 + new_col
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] =
new_state[new_blank_idx], new_state[blank_idx]
            neighbors.append((tuple(new_state), move_name))

    return neighbors

def dls(state, goal, depth, max_depth, visited, path):
    """Depth-Limited Search helper function."""
    if state == goal:
        return path

    if depth >= max_depth:
        return None

    visited.add(state)

    for neighbor, move in get_neighbors(state):
        if neighbor not in visited:
            result = dls(neighbor, goal, depth + 1, max_depth, visited,
path + [(neighbor, move)])
            if result is not None:
                return result

    visited.remove(state)
    return None

def iddfs_solve(start_state, goal_state, max_search_depth=50):
    """Solves the 8-puzzle using Iterative Deepening DFS to guarantee
    the shortest path."""
    for depth in range(max_search_depth + 1):
        visited = set()

```

```

        result = dls(start_state, goal_state, 0, depth, visited, [])
        if result is not None:
            return result
    return None

def print_board(state):
    """Prints a 1D tuple state as a 3x3 grid layout."""
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]}")
    print()

if __name__ == "__main__":
    goal = (1, 2, 3,
            4, 5, 6,
            7, 8, 0)

    start = (1, 2, 3,
            0, 4, 6,
            7, 5, 8)

    print(" Initial State Grid:")
    print_board(start)

    solution_path = iddfs_solve(start, goal)

    if solution_path is not None:
        print(f" Success! Optimal solution found with a path cost of:
{len(solution_path)}\n")

        current_state = start
        for step_idx, (next_state, move) in enumerate(solution_path,
1):
            print(f"Step {step_idx}: Move blank space '{move}'")
            print_board(next_state)
        else:
            print(" No solution found within the maximum depth limit.")
    print("Dehini 1WN24CS089")

```

*** Initial State Grid:

```
1 2 3
0 4 6
7 5 8
```

Success! Optimal solution found with a path cost of: 3

Step 1: Move blank space 'Right'

```
1 2 3
4 0 6
7 5 8
```

Step 2: Move blank space 'Down'

```
1 2 3
4 5 6
7 0 8
```

Step 3: Move blank space 'Right'

```
1 2 3
4 5 6
7 8 0
```

Dehini 1WN24CS089